RegReportX 2.0 — Complete Project Walkthrough

Architecture Overview

```
Frontend (Angular 21 — port 4200)
       ↕ HTTP + JWT Bearer Token
Backend (Spring Boot — port 9090)
       ↕ JPA/Hibernate
Database (MySQL — localhost:3306/kishore)
```

Explanation:
RegReportX is a full-stack Regulatory Reporting application. The Angular frontend runs on port 4200 and communicates with the Spring Boot backend on port 9090 using HTTP REST API calls. Every call after login carries a JWT (JSON Web Token) in the Authorization header. The backend uses JPA (Java Persistence API) with Hibernate as the ORM (Object Relational Mapper) to talk to a MySQL database named `kishore`. This three-tier architecture keeps the UI, business logic, and data completely separated.

PHASE 0 — Application Bootstrap

Backend: RegreportxApplication.java

```
@SpringBootApplication
public class RegreportxApplication {
    public static void main(String[] args) {
        SpringApplication.run(RegreportxApplication.class, args);
    }
}
```

Explanation:
This is the entry point of the entire Spring Boot backend.

- `@SpringBootApplication` is a convenience annotation that combines three annotations in one:
  - `@Configuration` — marks this class as a source of Spring bean definitions.
  - `@EnableAutoConfiguration` — tells Spring Boot to automatically configure beans based on the JARs present in the classpath (e.g., it sees `spring-boot-starter-data-jpa` and auto-configures Hibernate and a DataSource).
  - `@ComponentScan` — tells Spring to scan all sub-packages of `com.cts.regreportx` and register every class annotated with `@Service`, `@Repository`, `@Controller`, `@Component`, etc. as Spring-managed beans.
- `SpringApplication.run(...)` bootstraps the embedded Tomcat server, loads all beans, runs `@PostConstruct` initializers, and starts listening on port 9090.

Backend: application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/kishore?createDatabaseI
spring.datasource.username=root
spring.datasource.password=root
server.port=9090
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
jwt.secret=4qhq8LrEBcYcaRHhx1TDmPQxyGWl24J1U+P6/WfW7OQ=
```

Explanation:
This file configures the runtime environment of the Spring Boot application.

- `spring.datasource.url` — The JDBC connection string. `createDatabaseIfNotExist=true` means MySQL will automatically create the database `kishore` if it does not already exist, so you never need to manually run `CREATE DATABASE`.
- `spring.datasource.username` / `password` — MySQL credentials used by HikariCP (Spring Boot's default connection pool).
- `server.port=9090` — The embedded Tomcat listens on port 9090 instead of the default 8080, so there is no port conflict with other local services.
- `spring.jpa.hibernate.ddl-auto=update` — Hibernate automatically compares the `@Entity` classes with the existing database schema and adds/alters columns as needed. It never drops data. Use `create-drop` for tests, `validate` for production.
- `spring.jpa.show-sql=true` — Prints every SQL query Hibernate generates to the console, which is useful for debugging N+1 query issues.

- `jwt.secret` — A Base64-encoded 256-bit secret key used by the JJWT library to sign and verify JWT tokens using the HMAC-SHA256 algorithm. It is injected into `JwtService` via `@Value("${jwt.secret}")`.

Frontend: main.ts

```
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

Explanation:
This is the Angular application's entry point, equivalent to `main()` in Java.

- `bootstrapApplication` is the standalone bootstrap API introduced in Angular 14+. It replaces the old `platformBrowserDynamic().bootstrapModule(AppModule)` approach — there is no `AppModule` or `NgModule` at all in this project.
- `App` is the root component (the shell that hosts `<router-outlet>`).
- `appConfig` is the application-level configuration object that registers global providers (router, HTTP client, interceptors).
- `.catch(err => console.error(err))` — If bootstrapping fails (e.g., wrong config, missing provider), the error is printed to the browser console instead of silently failing.

Frontend: app.config.ts

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

Explanation:
This is the central configuration for the Angular application, equivalent to `AppModule`'s `imports` and `providers` in the old module-based approach.

- `ApplicationConfig` is a TypeScript interface that defines the shape of a standalone app's configuration.

- `provideBrowserGlobalErrorListeners()` — Registers Angular's global error handler for unhandled browser errors and promise rejections.
- `provideRouter(routes)` — Registers the router and all the route definitions from `app.routes.ts`. This enables client-side navigation, lazy loading, and route guards.
- `provideHttpClient(withInterceptors([authInterceptor]))` — Registers Angular's `HttpClient` for making HTTP requests AND attaches the `authInterceptor` as a functional interceptor. Every single outgoing HTTP request will pass through `authInterceptor` automatically, which adds the JWT Bearer token. This replaces the old class-based `HTTP_INTERCEPTORS` multi-provider pattern.

PHASE 1 — Security Architecture (Before Any Request)

Backend: SecurityConfig.java

Annotation	Meaning
`@Configuration`	Spring bean factory class
`@EnableWebSecurity`	Activates Spring Security's web support
`@EnableMethodSecurity`	Enables `@PreAuthorize` on methods

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    return http
        .cors(cors -> cors.configurationSource(corsConfigurationSource())
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/v3/api-docs/**", "/swagger-ui/**").permitA
            .requestMatchers("/api/admin/**").hasAnyRole("ADMIN", "REGTEC
            .anyRequest().authenticated()
        )
        .exceptionHandling(ex -> ex
            .authenticationEntryPoint(jwtAuthenticationEntryPoint)
            .accessDeniedHandler(customAccessDeniedHandler)
        )
        .sessionManagement(sess ->
            sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter
```

```
        .build();
}
```

Explanation:
This method defines the entire HTTP security policy of the application as a `SecurityFilterChain` — an ordered list of filters every incoming request passes through.

- `.cors(...)` — Enables Cross-Origin Resource Sharing so the Angular app on port 4200 can call the Spring backend on port 9090. Without this, browsers would block the requests.
- `.csrf(csrf -> csrf.disable())` — CSRF (Cross-Site Request Forgery) protection is disabled because this API is stateless (no cookies/sessions). JWT tokens in the Authorization header are not vulnerable to CSRF attacks.
- `.authorizeHttpRequests(...)` — Defines URL-level access rules:
  - `/api/auth/**` → `permitAll()` — The login endpoint must be accessible without a token, otherwise no one could ever log in.
  - `/swagger-ui/**` → `permitAll()` — Allows developers to test APIs via the Swagger UI without authentication.
  - `/api/admin/**` → `hasAnyRole("ADMIN", "REGTECH_ADMIN")` — Only users with these roles can access admin endpoints.
  - `.anyRequest().authenticated()` — Every other endpoint requires a valid JWT token.
- `.exceptionHandling(...)` — Registers custom handlers:
  - `jwtAuthenticationEntryPoint` — Returns a JSON `401 Unauthorized` response when a request has no token or an invalid token, instead of Spring's default HTML error page.
  - `customAccessDeniedHandler` — Returns a JSON `403 Forbidden` response when a valid user tries to access a resource they don't have the role for.
- `SessionCreationPolicy.STATELESS` — Tells Spring Security to never create or use an HttpSession. Each request is completely self-contained with its JWT. This is essential for scalable REST APIs.
- `.addFilterBefore(authFilter, UsernamePasswordAuthenticationFilter.class)` — Inserts the custom `JwtAuthenticationFilter` into the filter chain before Spring's own username/password filter so it runs first on every request.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
```

```
public AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider(us
    provider.setPasswordEncoder(passwordEncoder());
    return provider;
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(List.of("http://localhost:4200"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE
    configuration.setAllowedHeaders(List.of("*"));
    configuration.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfiguratic
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
```

Explanation:
These three `@Bean` methods define supporting components for the security chain:

- `BCryptPasswordEncoder` — BCrypt is a one-way adaptive hashing algorithm specifically designed for passwords. It automatically salts each password (preventing rainbow table attacks) and the cost factor makes brute-force attacks extremely slow. When a user logs in, Spring calls `encoder.matches(rawPassword, storedHash)` — it never decrypts. Passwords are stored as BCrypt hashes in the `users` table.
- `DaoAuthenticationProvider` — This is Spring Security's standard provider that handles the login process. It receives a username+password, calls `CustomUserDetailsService.loadUserByUsername()` to fetch the user from DB, then uses `BCryptPasswordEncoder.matches()` to verify the password. If both checks pass, authentication succeeds.
- `CorsConfigurationSource` — Defines which origins, HTTP methods, and headers are allowed for CORS. `setAllowedOrigins(List.of("http://localhost:4200"))` whitelists only the Angular dev server. `setAllowCredentials(true)` allows cookies and Authorization headers to be sent cross-origin. `OPTIONS` method is included to support the CORS preflight requests browsers send before PUT/DELETE/POST.

PHASE 2 — Login Flow

Frontend: auth.model.ts

```
export interface AuthRequest  { username: string; password: string; }
export interface AuthResponse { token: string; }
```

Explanation:
These are TypeScript interfaces — they are purely compile-time type definitions and generate no JavaScript at runtime.

- `AuthRequest` defines the shape of the login payload sent to the backend. The `username` and `password` fields match exactly what the backend's `AuthRequest` DTO expects in its JSON body.
- `AuthResponse` defines what the backend returns upon successful login — a single `token` field containing the JWT string. TypeScript's generic `http.post<AuthResponse>(...)` uses this interface to give the response strong typing, enabling autocompletion and catching typos at compile time.
- `export` makes these interfaces importable in other files like `auth.service.ts` and `login.ts`.

Frontend: login.ts

```
@Component({ standalone: true, imports: [FormsModule, RouterLink], ... })
export class LoginComponent {
  username = '';
  password = '';
  errorMessage = '';
  isLoading = false;

  constructor(
    private authService: AuthService,
    private router: Router,
    private cdr: ChangeDetectorRef,
    private ngZone: NgZone
  ) {}

  private run(fn: () => void): void {
    this.ngZone.run(() => {
      fn();
      this.cdr.detectChanges();
    });
  }
```

```
  onSubmit(): void {
    if (!this.username || !this.password) {
      this.errorMessage = 'Please enter both username and password.';
      return;
    }
    this.isLoading = true;
    this.errorMessage = '';

    this.authService.login({ username: this.username, password: this.pass
      .subscribe({
        next: () => {
          this.run(() => {
            this.isLoading = false;
            const role = this.authService.getRole();
            switch (role) {
              case 'REGTECH_ADMIN':       this.router.navigate(['/admin']
              case 'COMPLIANCE_ANALYST':  this.router.navigate(['/complia
              case 'RISK_ANALYST':        this.router.navigate(['/risk'])
              case 'REPORTING_OFFICER':   this.router.navigate(['/reporti
              case 'OPERATIONS_OFFICER':  this.router.navigate(['/operati
              default:                    this.router.navigate(['/']); br
            }
          });
        },
        error: (err) => {
          this.run(() => {
            this.isLoading = false;
            if (err.status === 401 || err.status === 403) {
              this.errorMessage = err.error?.message || 'Invalid username
            } else if (err.status === 0) {
              this.errorMessage = 'Unable to connect to server. Please tr
            } else {
              this.errorMessage = 'Login failed. Incorrect Username or Pa
            }
          });
        }
      });
  }
}
```

Explanation:

This is the Login page component. Let's go through every part:

- `@Component({ standalone: true })` — Declares this as a standalone component, meaning it does not belong to any `NgModule`. It manages its own imports.
- `imports: [FormsModule, RouterLink]` — `FormsModule` brings in Angular's template-driven form directives (`ngModel`, `ngForm`, `ngSubmit`). `RouterLink` allows `[routerLink]` directives in the HTML template for navigation links.
- Class properties (`username`, `password`, `errorMessage`, `isLoading`) — These are bound to the HTML template via two-way `[(ngModel)]` binding. When the user types in the input fields, these values update automatically.
- Constructor injection — Angular's Dependency Injection (DI) system injects `AuthService` (for the login API call), `Router` (for programmatic navigation), `ChangeDetectorRef` (to manually trigger UI updates), and `NgZone` (to bring HTTP callbacks back into Angular's change detection zone).
- `private run(fn)` — This helper method wraps the callback inside `ngZone.run()` and then calls `cdr.detectChanges()`. This fixes a common bug where HTTP observable callbacks run outside Angular's zone, causing the UI (spinner, error message) to not update until the next user interaction.
- `onSubmit()` — Called when the login form is submitted:
  1. Validates that both fields are filled.
  2. Sets `isLoading = true` to show a spinner.
  3. Calls `authService.login()` which returns an `Observable`.
  4. `.subscribe({ next, error })` — subscribes to the Observable. `next` runs on success, `error` runs on failure.
  5. On success: reads the user's role from the decoded JWT and navigates to the role-specific dashboard using `router.navigate()`.
  6. On error: checks the HTTP status code — `401` / `403` means bad credentials, `0` means the backend is down (CORS/network error), anything else is a generic failure.

Frontend: auth.service.ts

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private readonly API_URL = 'http://localhost:9090/api/auth';
  private readonly TOKEN_KEY = 'jwt_token';

  constructor(private http: HttpClient) {}

  login(credentials: AuthRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>(`${this.API_URL}/login`, credenti
```

```
    tap(response => {
      if (response.token) {
        localStorage.setItem(this.TOKEN_KEY, response.token);
      }
    })
  );
}

logout(): void {
  localStorage.removeItem(this.TOKEN_KEY);
}

getToken(): string | null {
  return localStorage.getItem(this.TOKEN_KEY);
}

private decodeToken(): any {
  const token = this.getToken();
  if (!token) return null;
  try {
    return JSON.parse(atob(token.split('.')[1]));
  } catch {
    return null;
  }
}

isAuthenticated(): boolean {
  const payload = this.decodeToken();
  if (!payload) return false;
  return payload.exp * 1000 > Date.now();
}

getUsername(): string | null {
  return this.decodeToken()?.sub || null;
}

getRole(): string | null {
  return this.decodeToken()?.role || null;
}

hasRole(role: string): boolean {
  return this.isAuthenticated() && this.getRole() === role;
}
```

```
  hasAnyRole(...roles: string[]): boolean {
    return this.isAuthenticated() && roles.includes(this.getRole() || '')
  }
}
```

Explanation:
`AuthService` is the single source of truth for authentication state in the Angular app. It is a singleton (one instance app-wide) because of `providedIn: 'root'`.

- `API_URL` and `TOKEN_KEY` are `private readonly` constants — they cannot be changed after initialization and cannot be accessed outside this class.
- `login(credentials)` — Sends an HTTP POST request with the `AuthRequest` JSON body to `/api/auth/login`. Returns an `Observable<AuthResponse>`. The `.pipe(tap(...))` attaches a side-effect: when the response arrives, it saves the token to `localStorage` under the key `jwt_token`. `tap` does not modify the stream — it just observes. The Observable is returned to the caller so they can subscribe and handle navigation.
- `logout()` — Removes the token from `localStorage`. After this, all subsequent `isAuthenticated()` calls return `false` and the interceptor stops attaching the header.
- `getToken()` — Simple getter that reads the token string from `localStorage`. Called by the `authInterceptor` on every request.
- `decodeToken()` — A JWT has three Base64URL-encoded parts separated by dots: `header.payload.signature`. This method splits the token, takes the middle part (index `[1]`), decodes it using the native browser `atob()` function, and parses the JSON payload. The `try/catch` prevents crashes on malformed tokens. This is client-side decoding only — no signature verification happens here (that is done on the backend).
- `isAuthenticated()` — Reads the `exp` (expiration) claim from the decoded payload. `exp` is a Unix timestamp in seconds; multiplying by 1000 converts to milliseconds for comparison with `Date.now()`. Returns `true` only if the token exists AND has not expired.
- `getRole()` — Reads the `role` claim that the backend embedded in the token during `JwtService.generateToken()`. This is tamper-proof because the JWT is signed — if someone modifies the role in localStorage, the backend's `JwtAuthenticationFilter` will reject the tampered token.
- `hasRole()` / `hasAnyRole()` — Convenience methods used by route guards to check roles without repeating the `isAuthenticated()` check.

Backend: AuthController.java

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtService jwtService;

    @Autowired
    private UserRepository userRepository;

    @PostMapping("/login")
    public ResponseEntity<?> authenticateAndGetToken(@RequestBody AuthRec
        Authentication authentication = authenticationManager.authenticat
            new UsernamePasswordAuthenticationToken(
                authRequest.getUsername(), authRequest.getPassword())
        );

        if (authentication.isAuthenticated()) {
            User user = userRepository.findByUsername(authRequest.getUser
                .orElseThrow(() -> new UsernameNotFoundException("User no

            if (!"ACTIVE".equalsIgnoreCase(user.getStatus())) {
                return ResponseEntity.status(403)
                    .body(Map.of("message", "User is not currently active
            }

            String token = jwtService.generateToken(user.getUsername(), u
            return ResponseEntity.ok(Map.of("token", token));
        } else {
            throw new UsernameNotFoundException("Invalid user request!");
        }
    }
}
```

Explanation:
This controller handles the one public endpoint of the entire application — the login API.

- `@RestController` — Combines `@Controller` and `@ResponseBody`. Every method return value is automatically serialized to JSON and written to the HTTP response body.

- `@RequestMapping("/api/auth")` — All endpoints in this controller are prefixed with `/api/auth`.
- `@Autowired` — Spring injects the declared beans (AuthenticationManager, JwtService, UserRepository) at startup. This is constructor injection via field injection (for brevity in this controller).
- `@PostMapping("/login")` — Maps HTTP POST requests to `/api/auth/login` to this method. This URL is in the `permitAll()` list in `SecurityConfig`, so no token is needed.
- `@RequestBody AuthRequest authRequest` — Spring uses Jackson (the JSON library) to deserialize the incoming JSON body `{ "username": "...", "password": "..." }` into an `AuthRequest` object.
- `authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(...))` — This triggers the full Spring Security authentication chain: it calls `CustomUserDetailsService.loadUserByUsername()` to load the user from DB, then `BCryptPasswordEncoder.matches()` to verify the password. If either fails, Spring throws `BadCredentialsException` which returns HTTP 401.
- `"ACTIVE".equalsIgnoreCase(user.getStatus())` — A second layer of business-level check beyond Spring Security. Even if the credentials are correct, an inactive/suspended user is rejected with HTTP 403. Note: `"ACTIVE".equalsIgnoreCase(...)` is written this way (literal first) to prevent a `NullPointerException` if `status` is null.
- `jwtService.generateToken(user.getUsername(), user.getRole())` — Generates a signed JWT with the username as the subject and the role as a custom claim.
- `ResponseEntity.ok(Map.of("token", token))` — Returns HTTP 200 with JSON body `{ "token": "eyJ..." }`.

Backend: JwtService.java

```
@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secret;

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsRes
        final Claims claims = extractAllClaims(token);
```

```
    return claimsResolver.apply(claims);
}

public String generateToken(String username, String role) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("role", role);
    return createToken(claims, username);
}

private String createToken(Map<String, Object> claims, String userNam
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(userName)
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 6
        .signWith(getSignKey(), SignatureAlgorithm.HS256)
        .compact();
}

private Claims extractAllClaims(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(getSignKey())
        .build()
        .parseClaimsJws(token)
        .getBody();
}

private Key getSignKey() {
    byte[] keyBytes = io.jsonwebtoken.io.Decoders.BASE64.decode(secre
    return Keys.hmacShaKeyFor(keyBytes);
}

public Boolean validateToken(String token, UserDetails userDetails) {
    final String username = extractUsername(token);
    return (username.equalsIgnoreCase(userDetails.getUsername()) && !
}

private Boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());
}

public Date extractExpiration(String token) {
    return extractClaim(token, Claims::getExpiration);
```

```
    }
}
```

Explanation:

`JwtService` is the single responsible class for all JWT operations — creation, parsing, and validation.

- `@Value("${jwt.secret}")` — Spring reads the `jwt.secret` property from `application.properties` and injects the value into this field at startup. The secret never appears hardcoded in Java source code.
- `generateToken(username, role)` — Creates a `Map` of extra claims (currently just `role`) and calls `createToken`. The role is embedded in the JWT payload so the frontend can read it without another API call.
- `createToken(claims, userName)` — Uses the JJWT `Jwts.builder()` fluent API to construct the JWT:
  - `setClaims(claims)` — Sets the custom payload data (includes `role`).
  - `setSubject(userName)` — Sets the standard `sub` claim to the username. This is the primary identity claim.
  - `setIssuedAt(new Date(...))` — Sets the `iat` (issued at) claim to the current time.
  - `setExpiration(... + 1000 * 60 * 60 * 10)` — Sets the `exp` claim to 10 hours from now (1000ms × 60s × 60min × 10h). After this, the token is automatically rejected.
  - `signWith(getSignKey(), SignatureAlgorithm.HS256)` — Signs the header+payload with the HMAC-SHA256 algorithm using the secret key. This produces the third segment of the JWT. Any tampering with the header or payload will make signature verification fail.
  - `.compact()` — Serializes everything into the final `xxxxx.yyyyy.zzzzz` JWT string.
- `getSignKey()` — Decodes the Base64-encoded secret string into raw bytes and creates a `SecretKey` object using `Keys.hmacShaKeyFor()`. This must produce the exact same key for both signing and verification.
- `extractAllClaims(token)` — Uses `Jwts.parserBuilder()` to parse and verify the token. If the signature is invalid or the token is expired, JJWT throws an exception that is caught upstream.
- `extractClaim(token, claimsResolver)` — Generic method using a `Function<Claims, T>` function reference. For example, `Claims::getSubject` is a method reference that extracts the `sub` field. This pattern avoids code duplication across multiple claim extraction methods.
- `validateToken(token, userDetails)` — Final validation: checks that the username in the token matches the loaded user AND that the token is not expired. Both conditions must be true.

Backend: CustomUserDetailsService.java

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws Usernam
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(
                "User not found with username: " + username));

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            List.of(new SimpleGrantedAuthority("ROLE_" + user.getRole()))
        );
    }
}
```

Explanation:
This class is the bridge between Spring Security and your database. Spring Security does not know about your `User` entity — it only knows the `UserDetails` interface.

- `implements UserDetailsService` — This interface has one method: `loadUserByUsername(String username)`. Spring Security calls it whenever it needs to authenticate a user (during login) or re-authenticate a request (via `JwtAuthenticationFilter`).
- `userRepository.findByUsername(username)` — Queries the database for the user by username. Returns an `Optional<User>`.
- `.orElseThrow(() -> new UsernameNotFoundException(...))` — If no user is found, throws `UsernameNotFoundException` which Spring Security converts to a `401 Unauthorized` response.
- `new org.springframework.security.core.userdetails.User(...)` — Constructs Spring Security's own `UserDetails` object (note: this is a different class from your `com.cts.regreportx.model.User`). It takes:
    1. The username string.

2. The BCrypt-hashed password (Spring Security will call `passwordEncoder.matches()` on this).
3. A list of `GrantedAuthority` objects representing the user's roles/permissions.

- `new SimpleGrantedAuthority("ROLE_" + user.getRole())` — Spring Security's `hasRole("REGTECH_ADMIN")` internally checks for an authority named `"ROLE_REGTECH_ADMIN"`. So if the DB stores `"REGTECH_ADMIN"`, the prefix `"ROLE_"` must be added here. This is a Spring Security convention.

Backend: User.java

```
@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(unique = true)
    private String username;

    @Column(unique = true)
    private String email;

    private String password;

    private String role;

    private String status;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        if (this.createdAt == null) {
            this.createdAt = LocalDateTime.now();
```

```
        }
    }
}
```

Explanation:
This is a JPA Entity class — it maps to the `users` table in MySQL. Hibernate reads this class and creates/updates the table schema automatically.

- `@Entity` — Marks this class as a JPA-managed persistent entity. Hibernate registers it and creates a `users` table.
- `@Table(name = "users")` — Explicitly names the table `users`. Without this, Hibernate would default to the class name `user` which is a reserved keyword in MySQL.
- `@Data` — Lombok annotation that auto-generates at compile time: `getUsername()`, `setUsername()`, `toString()`, `equals()`, `hashCode()` for all fields. This eliminates hundreds of lines of boilerplate.
- `@NoArgsConstructor` — Lombok generates a no-argument constructor. JPA requires a no-arg constructor to instantiate entities via reflection when loading from the database.
- `@Id` — Marks `id` as the primary key.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` — Tells the database (MySQL) to auto-increment the `id` column. When you call `repository.save(user)`, Hibernate inserts the row and MySQL assigns the next available ID.
- `@Column(unique = true)` — Adds a `UNIQUE` constraint to the column in MySQL, preventing two users from having the same `username` or `email`.
- `private String role` — Stores the plain role string (e.g., `"REGTECH_ADMIN"`). There is no separate `Role` entity — it is a simple string to keep the model flat.
- `private String status` — `"ACTIVE"` or `"INACTIVE"`. Used by `AuthController` to block deactivated users from logging in.
- `@PrePersist` — A JPA lifecycle callback that runs automatically just before the entity is first inserted into the database. It sets `createdAt` to the current timestamp if it hasn't been set, ensuring every user record has a creation timestamp.

Backend: UserRepository.java

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    Optional<User> findByUsername(String username);
    List<User> findByRole(String role);
}
```

Explanation:
This interface is a Spring Data JPA repository. It requires zero implementation code — Spring generates the SQL queries automatically at startup.

- `extends JpaRepository<User, Long>` — `User` is the entity type, `Long` is the primary key type. This inheritance gives you `save(user)`, `findById(id)`, `findAll()`, `deleteById(id)`, `count()`, and many more methods for free.
- `findByEmail(String email)` — Spring Data parses this method name and generates `SELECT * FROM users WHERE email = ?`. Returns `Optional<User>` to force callers to handle the "user not found" case explicitly, preventing NullPointerExceptions.
- `findByUsername(String username)` — Generates `SELECT * FROM users WHERE username = ?`. Used by `CustomUserDetailsService` during every request authentication.
- `findByRole(String role)` — Generates `SELECT * FROM users WHERE role = ?`. Used by `NotificationService` to find all users of a particular role to send them notifications.

Backend: JwtAuthenticationFilter.java

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtService jwtService;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");
        String token = null;
        String username = null;

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            token = authHeader.substring(7);
            try {
```

```
                username = jwtService.extractUsername(token);
            } catch (Exception e) {
                // Invalid token — username stays null, request proceeds
            }
        }

        if (username != null && SecurityContextHolder.getContext().getAut
            UserDetails userDetails = userDetailsService.loadUserByUserna

            if (jwtService.validateToken(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());
                authToken.setDetails(
                    new WebAuthenticationDetailsSource().buildDetails(req
                SecurityContextHolder.getContext().setAuthentication(auth
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

Explanation:

This filter runs on every single HTTP request that reaches the backend (except those matched by `permitAll()`). It is the gatekeeper that validates JWTs and populates the security context.

- `extends OncePerRequestFilter` — Guarantees this filter executes exactly once per HTTP request, even if the request is forwarded internally within the servlet container.
- `@Component` — Spring registers this as a bean, and `SecurityConfig` injects it into the filter chain via `addFilterBefore(authFilter, ...)`.
- `request.getHeader("Authorization")` — Reads the `Authorization` HTTP header that the frontend's `authInterceptor` attaches to every request.
- `authHeader.startsWith("Bearer ")` — The OAuth2/JWT convention is to prefix the token with `Bearer` (note the space). The token itself starts at index 7.
- `jwtService.extractUsername(token)` — Decodes the JWT and retrieves the `sub` (subject) claim which is the username. If the token is malformed or tampered with, JJWT throws an exception which is silently caught — `username` stays `null` and the request proceeds as unauthenticated.
- `SecurityContextHolder.getContext().getAuthentication() == null` — Checks that no authentication has already been set for this request (e.g., by a previous filter). Prevents double-authentication.

- `userDetailsService.loadUserByUsername(username)` — Loads the full `UserDetails` from the database to get authorities (roles) and the stored password hash.
- `jwtService.validateToken(token, userDetails)` — Verifies (1) the username in the token matches the loaded user and (2) the token is not expired.
- `new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities())` — Creates a fully authenticated token. The second argument `null` means no credentials (password) are stored in the security context — the JWT already proved identity. The third argument is the list of roles.
- `SecurityContextHolder.getContext().setAuthentication(authToken)` — Stores the authenticated principal in a thread-local `SecurityContext`. From this point forward in the request lifecycle, any code (service, controller, `@PreAuthorize`) can call `SecurityContextHolder.getContext().getAuthentication()` to find out who is making the request.
- `filterChain.doFilter(request, response)` — CRITICAL: Must always be called to pass the request to the next filter or the controller. Without this, the request would hang.

PHASE 3 — Route Guards (Frontend)

Frontend: app.routes.ts

```
export const routes: Routes = [
  { path: '',            loadComponent: () => import('./pages/home/home').
  { path: 'login',       loadComponent: () => import('./pages/login/login'
  { path: 'unauthorized', loadComponent: () => import('./pages/unauthoriz
  { path: 'admin',       loadComponent: () => import('./pages/admin/admin'
  { path: 'compliance', loadComponent: () => import('./pages/compliance/c
  { path: 'risk',        loadComponent: () => import('./pages/risk/risk').
  { path: 'reporting',   loadComponent: () => import('./pages/reporting/re
  { path: 'operations', loadComponent: () => import('./pages/operations/c
  { path: '**',          redirectTo: '' }
];
```

Explanation:
This file defines all client-side routes (pages) in the Angular application.

- `Routes` is an array of `Route` objects. The Angular Router matches the browser URL against each `path` in order.

- `loadComponent: () => import(...).then(m => m.ComponentClass)` — This is Angular's **lazy loading** syntax. The component's JavaScript bundle is NOT downloaded when the app first loads. It is downloaded on-demand only when the user navigates to that route. This dramatically reduces initial load time. The `import()` is a standard JavaScript dynamic import.
- `canActivate: [adminGuard]` — Before navigating to this route, Angular calls the `adminGuard` function. If it returns `true`, navigation proceeds. If it returns `false`, navigation is blocked and the user is redirected. Multiple guards can be listed — all must return `true`.
- `{ path: '**', redirectTo: '' }` — Wildcard route that catches any URL that does not match any defined route (e.g., `/gibberish`) and redirects to the home page. Must be the last route in the array.
- Routes without `canActivate` (`login`, `unauthorized`, `''`) are publicly accessible. Authenticated users still see them, but guards handle redirecting away if already logged in.

Frontend: auth.guard.ts

```
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.isAuthenticated()) return true;
  router.navigate(['/login']);
  return false;
};

export const adminGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.hasRole('REGTECH_ADMIN')) return true;
  if (!authService.isAuthenticated()) router.navigate(['/login']);
  else router.navigate(['/unauthorized']);
  return false;
};

export const complianceGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.hasAnyRole('COMPLIANCE_ANALYST', 'REGTECH_ADMIN')) retu
  if (!authService.isAuthenticated()) router.navigate(['/login']);
  else router.navigate(['/unauthorized']);
  return false;
};
```

```
// riskGuard, reportingGuard, operationsGuard follow the same pattern
```

Explanation:
Guards are the frontend's access control layer — they mirror the backend's `@PreAuthorize` rules but at the navigation level.

- `CanActivateFn` — The functional type signature for a route guard in Angular 15+. It replaces the old class-based `CanActivate` interface. A `CanActivateFn` is simply a function that returns `boolean | UrlTree | Observable<boolean> | Promise<boolean>`.
- `inject(AuthService)` — Angular's `inject()` function provides dependency injection outside of a class constructor. It can be used in any injection context, including standalone functions and guards. This is more tree-shakeable than class-based injection.
- `authService.isAuthenticated()` — Checks the JWT expiry client-side (fast, no HTTP call). If the token has expired, the user is sent to `/login`.
- Two-branch redirect logic in role guards:
  - Not authenticated at all → `/login` (the user needs to log in first).
  - Authenticated but wrong role → `/unauthorized` (the user is logged in but doesn't have permission).
- `adminGuard` requires exactly `REGTECH_ADMIN`. `complianceGuard` allows either `COMPLIANCE_ANALYST` or `REGTECH_ADMIN` (admin can access all areas). This hierarchy is consistent with the backend's `@PreAuthorize` annotations.
- These guards run synchronously (no HTTP calls), so navigation is instant with no loading delay.

Frontend: auth.interceptor.ts

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const router = inject(Router);
  const token = authService.getToken();

  let authReq = req;
  if (token) {
    authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }

  return next(authReq).pipe(
    catchError((error: HttpErrorResponse) => {
```

```
      if (error.status === 401) {
        authService.logout();
        router.navigate(['/login']);
      }
      return throwError(() => error);
    })
  );
};
```

Explanation:
The HTTP interceptor is middleware that runs for every outgoing HTTP request in the Angular application. It is registered globally in `app.config.ts` via `withInterceptors([authInterceptor])`.

- `HttpInterceptorFn` — The functional type for interceptors in Angular 15+. `req` is the outgoing `HttpRequest` and `next` is the function that passes the request to the next interceptor or to the actual HTTP layer.
- `req.clone({ setHeaders: { Authorization: \` Bearer ${token}` } })` — `HTTP requests in Angular are immutable (` HttpRequest `objects cannot be modified). To change them, you must clone the object with the modifications. This line creates a new request identical to the original but with an extra` Authorization `header. The` Bearer ` prefix is the standard OAuth2/JWT convention.
- `next(authReq)` — Passes the cloned, modified request through the pipeline. Returns an `Observable<HttpEvent>`.
- `.pipe(catchError(...))` — Catches errors from the HTTP response. If the backend returns `401 Unauthorized` (e.g., token expired mid-session), the interceptor automatically:
  1. Calls `authService.logout()` to clear the stale token from `localStorage`.
  2. Navigates to `/login` to force re-authentication.
- `throwError(() => error)` — Re-emits the error so the calling service/component can still handle it in its own `subscribe({ error: ... })` block if needed.

PHASE 4 — Data Ingestion (Operations Officer)

Backend: CsvUploadController.java

```
@RestController
@RequestMapping("/api/ingestion/csv")
```

```
@PreAuthorize("hasAnyRole('OPERATIONS_OFFICER', 'REGTECH_ADMIN')")
public class CsvUploadController {

    private final CsvUploadService csvUploadService;
    private final NotificationService notificationService;

    public CsvUploadController(CsvUploadService csvUploadService,
                               NotificationService notificationService)
        this.csvUploadService = csvUploadService;
        this.notificationService = notificationService;
    }

    @PostMapping("/deposits/upload")
    public ResponseEntity<UploadResponse> uploadDeposits(
            @RequestParam("file") MultipartFile file) {
        UploadResponse response = csvUploadService.uploadDeposits(file);
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "New deposits data uploaded — ready for ingestion", "Data Upl
        return ResponseEntity.ok(response);
    }

    @PostMapping("/loans/upload")
    public ResponseEntity<UploadResponse> uploadLoans(@RequestParam("file
        UploadResponse response = csvUploadService.uploadLoans(file);
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "New loans data uploaded — ready for ingestion", "Data Uploac
        return ResponseEntity.ok(response);
    }

    @PostMapping("/general-ledger/upload")
    public ResponseEntity<UploadResponse> uploadGeneralLedger(@RequestPar
        UploadResponse response = csvUploadService.uploadGeneralLedger(fi
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "New general ledger data uploaded — ready for ingestion", "Da
        return ResponseEntity.ok(response);
    }

    @PostMapping("/treasury/upload")
    public ResponseEntity<UploadResponse> uploadTreasury(@RequestParam("f
        UploadResponse response = csvUploadService.uploadTreasury(file);
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "New treasury data uploaded — ready for ingestion", "Data Upl
        return ResponseEntity.ok(response);
```

```
    }
}
```

Explanation:
This controller handles CSV file uploads for four types of source data: deposits, loans, general ledger entries, and treasury trades.

- `@PreAuthorize("hasAnyRole('OPERATIONS_OFFICER', 'REGTECH_ADMIN')")` — This class-level annotation protects ALL four endpoints. Only users with these roles can call them. The check runs before the method body executes. This is enforced by Spring AOP (Aspect-Oriented Programming) which wraps the method in a proxy that checks the role first.
- Constructor injection (`public CsvUploadController(...)`) — This is the recommended injection style over `@Autowired` field injection. It makes dependencies explicit, enables immutability (`final` fields), and makes unit testing easier (no Spring context needed, just pass mocks).
- `@RequestParam("file") MultipartFile file` — `MultipartFile` is Spring's abstraction over an uploaded file from a `multipart/form-data` HTTP request. The `"file"` string must match the form field name in the HTTP request (and in the frontend's `FormData.append('file', ...)`).
- The four endpoints follow the exact same pattern: parse the CSV → save records → notify Compliance Analyst. The notification creates a persistent `Notification` record in the DB so the COMPLIANCE_ANALYST sees an in-app alert.
- `ResponseEntity<UploadResponse>` — Returns HTTP 200 with an `UploadResponse` object that contains details like how many records were parsed and saved.

Backend: DataIngestionService.java

```
@Service
public class DataIngestionService {

    @PostConstruct
    public void initDataSources() {
        if (dataSourceRepository.count() == 0) {
            String[] systemNames = { "Loan System", "Deposit System",
                                     "Treasury System", "GL System" };
            for (String name : systemNames) {
                DataSource source = new DataSource();
                source.setName(name);
                source.setSourceType("DATABASE");
                source.setStatus("ACTIVE");
```

```
                dataSourceRepository.save(source);
            }
        }
    }

    public List<RawDataBatch> runIngestion() {
        List<RawDataBatch> batches = new ArrayList<>();
        List<DataSource> sources = dataSourceRepository.findAll();

        for (DataSource source : sources) {
            int rowCount = 0;
            String resourceName = source.getName().toLowerCase();

            if (resourceName.contains("loan"))     rowCount = sourceDataS
            else if (resourceName.contains("deposit")) rowCount = sourceI
            else if (resourceName.contains("treasury")) rowCount = source
            else if (resourceName.contains("gl"))  rowCount = sourceDataS

            RawDataBatch batch = new RawDataBatch();
            batch.setSource(source);
            batch.setIngestedDate(LocalDateTime.now());
            batch.setRowCount(rowCount);
            batch.setStatus("Ingested");
            batches.add(rawDataBatchRepository.save(batch));

            auditService.logAction("RUN_INGESTION", resourceName);
        }

        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "Data ingestion completed — ready for validation", "Ingestior
        return batches;
    }

    public List<RawDataBatch> getAllBatches() {
        return rawDataBatchRepository.findAll();
    }
}
```

Explanation:

This service orchestrates the data ingestion process — it reads the already-uploaded source data and creates `RawDataBatch` records that act as a snapshot of how much data was ingested from each system at a point in time.

- `@PostConstruct` on `initDataSources()` — This method runs automatically right after Spring finishes creating and injecting all dependencies into this bean. It seeds the `DataSource` table with four entries (`Loan System` , `Deposit System` , `Treasury System` , `GL System`) if none exist yet. This is a one-time initialization, gated by `count() == 0` , so it never creates duplicates on restarts.
- `runIngestion()` — Triggered by `POST /api/ingestion/run` . It loops over all `DataSource` records, queries the actual uploaded data for each source, counts the rows, and creates a `RawDataBatch` record with status `"Ingested"` and a timestamp. Each `RawDataBatch` represents one ingestion snapshot of one data source.
- `auditService.logAction("RUN_INGESTION", resourceName)` — Records an audit log entry for every ingested source. This creates a tamper-evident trail: the admin can see exactly when and by whom each data source was ingested.
- `notificationService.notifyRole("COMPLIANCE_ANALYST", ...)` — After all sources are ingested, notifies all active COMPLIANCE_ANALYST users that ingestion is complete and they can proceed with data quality validation.

PHASE 5 — Data Quality (Compliance Analyst)

Backend: DataQualityIssue.java

```
@Entity
@Table(name = "DataQualityIssue")
@Data
@NoArgsConstructor
public class DataQualityIssue {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "IssueID")
    private Integer issueId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "BatchID")
    private RawDataBatch batch;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "RuleID")
    private ValidationRule rule;
```

```
    @Column(name = "RecordID")
    private String recordId;

    @Column(name = "Message")
    private String message;

    @Column(name = "Severity")
    private String severity;

    @Column(name = "LoggedDate")
    private LocalDateTime loggedDate;

    @Column(name = "Status")
    private String status;
}
```

Explanation:
This entity represents a single data quality problem found in the ingested data.

- `@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "BatchID")` — Each issue belongs to one `RawDataBatch` (many issues can come from one batch). `FetchType.LAZY` means Hibernate will NOT automatically load the entire `RawDataBatch` object when a `DataQualityIssue` is fetched. The batch is loaded only when `issue.getBatch()` is explicitly called. This is crucial for performance — without LAZY, loading 1000 issues would trigger 1000 additional SQL queries for each batch.
- `@ManyToOne @JoinColumn(name = "RuleID")` — Links the issue to the specific `ValidationRule` that was violated. This lets the system know which field and condition caused the problem.
- `recordId` — The ID of the specific `RawRecord` that failed validation, stored as a String for flexibility across different record types.
- `message` — Human-readable description of the problem (e.g., `"loanAmount cannot be negative: -500.00"`).
- `severity` — `"High"`, `"Medium"`, or `"Critical"`. Used by the frontend to color-code issues and help the analyst prioritize.
- `status` — `"Open"` means the issue still needs resolution. `"Resolved"` means a COMPLIANCE_ANALYST has reviewed and corrected it.

Backend: DataQualityController.java → DataQualityService.java

```
// Controller
@RestController
@RequestMapping("/api/data-quality")
@PreAuthorize("hasAnyRole('COMPLIANCE_ANALYST', 'REGTECH_ADMIN')")
public class DataQualityController {

    @GetMapping("/issues")
    @PreAuthorize("hasAnyRole('COMPLIANCE_ANALYST', 'REGTECH_ADMIN', 'REP
    public ResponseEntity<List<DataQualityIssueDTO>> getOpenIssues() {
        List<DataQualityIssue> issues = dataQualityService.getOpenIssues(
        auditService.logAction("VIEWED_OPEN_QUALITY_ISSUES", "DataQuality
            "Open issues: " + issues.size());
        List<DataQualityIssueDTO> dtos = issues.stream()
            .map(issue -> modelMapper.map(issue, DataQualityIssueDTO.clas
            .collect(Collectors.toList());
        return ResponseEntity.ok(dtos);
    }

    @PutMapping("/issues/{id}/resolve")
    public ResponseEntity<?> resolveIssue(@PathVariable Integer id,
                                          @RequestBody DataQualityResolv
        Map<String, Object> resolvedData = dataQualityService.resolveIssu
        auditService.logAction("RESOLVED_QUALITY_ISSUE", "DataQuality",
            "Issue #" + id + " | Justification: " + request.getJustificat
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "Data quality issue #" + id + " resolved", "Data Quality");
        notificationService.notifyRole("REPORTING_OFFICER",
            "Data quality issue #" + id + " resolved — clean data availab
        return ResponseEntity.ok(Map.of(
            "message", "Issue resolved successfully and CorrectionLog ger
            "issue", resolvedData.get("issue"),
            "patchedRecordPayload", resolvedData.get("patchedRecordPayloa
        ));
    }
}


// Service — resolve logic
@Transactional
public Map<String, Object> resolveIssue(Integer issueId, DataQualityResol
    DataQualityIssue issue = dataQualityIssueRepository.findById(issueId)
        .orElseThrow(() -> new ResourceNotFoundException("Data Quality Is

    if ("Resolved".equalsIgnoreCase(issue.getStatus()))
```

```
        throw new ValidationException("Issue is already resolved");

    // Patch the raw JSON payload of the offending RawRecord
    Integer rawRecordId = Integer.parseInt(issue.getRecordId());
    RawRecord rawRecord = rawRecordRepository.findById(rawRecordId).get()
    ValidationRule rule = issue.getRule();
    String fieldName = rule.getRuleExpression().split(" ")[0];
    ObjectNode jsonNode = (ObjectNode) objectMapper.readTree(rawRecord.ge
    jsonNode.put(fieldName, request.getCorrectedValue().trim());
    rawRecord.setPayloadJson(jsonNode.toString());
    rawRecordRepository.save(rawRecord);

    // Mark issue resolved
    issue.setStatus("Resolved");
    dataQualityIssueRepository.save(issue);

    // Create correction audit trail
    CorrectionLog log = new CorrectionLog();
    log.setOldValue(issue.getMessage());
    log.setNewValue("Corrected to " + request.getCorrectedValue()
        + " | Reason: " + request.getJustification());
    log.setCorrectedDate(LocalDateTime.now());
    Authentication auth = SecurityContextHolder.getContext().getAuthentic
    userRepository.findByUsername(auth.getName()).ifPresent(log::setCorre
    correctionLogRepository.save(log);

    return Map.of("issue", issue, "patchedRecordPayload", jsonNode.toStri
}
```

Explanation:
The data quality resolution flow involves the controller receiving the request and the service performing the actual database operations.

- `@GetMapping("/issues")` overrides the class-level `@PreAuthorize` to also allow `REPORTING_OFFICER` to view open issues (read-only). The resolve endpoint keeps the stricter class-level restriction (only `COMPLIANCE_ANALYST` or admin).
- `@PathVariable Integer id` — Extracts the `{id}` path variable from the URL (e.g., `/api/data-quality/issues/42/resolve` → `id = 42`).
- `modelMapper.map(issue, DataQualityIssueDTO.class)` — Converts the JPA entity to a DTO (Data Transfer Object). DTOs prevent exposing internal entity fields (like lazy-loaded relationships) directly in the API response, preventing serialization errors and data leaks.

- `.stream().map(...).collect(Collectors.toList())` — Java Streams pipeline: converts the `List<DataQualityIssue>` entity list into a `List<DataQualityIssueDTO>` in one fluent expression.
- `@Transactional` on `resolveIssue` — All database operations in this method (save `RawRecord`, save `DataQualityIssue`, save `CorrectionLog`) are wrapped in a single database transaction. If any one of them fails (e.g., DB constraint violation), all changes are rolled back automatically, keeping the database consistent.
- `ObjectMapper` / `ObjectNode` — Jackson's JSON library is used to parse the `RawRecord`'s `payloadJson` string into a mutable JSON tree (`ObjectNode`), update the offending field with the corrected value, then serialize it back to a string. This surgical patching avoids having to re-ingest the entire record.
- `CorrectionLog` — Creates a permanent, immutable audit trail of what was changed, the old value, the new value, the reason (justification), the timestamp, and which user made the correction. This is critical for regulatory compliance — every data correction must be traceable.

PHASE 6 — Risk Calculation (Risk Analyst)

Backend: RiskController.java → RiskCalculationService.java

```
// Controller
@RestController
@RequestMapping("/api/risk")
@PreAuthorize("hasAnyRole('RISK_ANALYST', 'REGTECH_ADMIN')")
public class RiskController {

    @GetMapping("/metrics")
    @PreAuthorize("hasAnyRole('RISK_ANALYST', 'REGTECH_ADMIN', 'REPORTING
    public ResponseEntity<List<RiskMetricDTO>> getMetrics() {
        List<RiskMetric> metrics = riskCalculationService.getAllMetrics()
        return ResponseEntity.ok(metrics.stream()
            .map(m -> modelMapper.map(m, RiskMetricDTO.class))
            .collect(Collectors.toList()));
    }

    @PostMapping("/calculate/{reportId}")
    public ResponseEntity<Map<String, Object>> calculateMetrics(
            @PathVariable Integer reportId) {
        List<RiskMetric> metrics = riskCalculationService.calculateMetric
        Map<String, Object> response = new HashMap<>();
        response.put("reportId", reportId);
```

```
        response.put("metricsCalculated",
            metrics.stream().map(RiskMetric::getMetricName).collect(Colle
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "Risk metrics ready for Report #" + reportId, "Risk");
        notificationService.notifyRole("REPORTING_OFFICER",
            "Risk metrics completed for draft Report #" + reportId, "Risk
        return ResponseEntity.ok(response);
    }
}


// Service
public List<RiskMetric> calculateMetrics(Integer reportId) {
    riskMetricRepository.deleteByReport_ReportId(reportId); // Clear stal

    RegReport report = reportRepository.findById(reportId)
        .orElseThrow(() -> new RuntimeException("Report Not Found"));

    if (!"DRAFT".equalsIgnoreCase(report.getStatus()))
        throw new RuntimeException("Risk metrics can only be calculated f

    // Aggregate raw source data
    Map<String, BigDecimal> context = new HashMap<>();
    context.put("Total_Loans",           /* sum all loan amounts */);
    context.put("Total_Deposits",         /* sum all deposit amounts */);
    context.put("Loan_to_Deposit_Ratio",  /* totalLoans / totalDeposits *
    context.put("Treasury_Exposure",      /* sum all treasury notionals *
    context.put("Net_GL_Balance",         /* sum(credit - debit) per GL e
    context.put("RWA",                    /* risk-weighted sum by loan ty
    context.put("MAX(Customer_Total_Load)", /* highest exposure by custom

    // Evaluate each template field formula
    for (TemplateField field : templateService.getFieldsByTemplateId(
            report.getTemplate().getTemplateId())) {
        BigDecimal value = riskEvaluator.evaluateFormula(
            field.getMappingExpression(), context);
        context.put(field.getFieldName(), value);
        metrics.add(createMetric(report, field.getFieldName(), value));
    }

    auditService.logAction("CALCULATE_RISK_METRICS", "ReportID: " + repor
        "Dynamically calculated " + metrics.size() + " metrics.");
    return metrics;
}
```

```
private BigDecimal getRiskWeight(String loanType) {
    switch (loanType.toLowerCase()) {
        case "home loan":     return new BigDecimal("0.50");
        case "personal loan":
        case "auto loan":     return new BigDecimal("0.75");
        case "corporate loan":
        default:              return new BigDecimal("1.00");
    }
}
```

Explanation:
Risk calculation is the analytical heart of the system. It transforms raw financial data into regulatory metrics.

- `riskMetricRepository.deleteByReport_ReportId(reportId)` — Clears any previously calculated metrics for this report before recalculating. This ensures idempotency — calling the endpoint multiple times always produces a fresh, correct result rather than accumulating duplicates.
- Status guard (`!"DRAFT"`) — Risk metrics are only meaningful for a report still in the DRAFT phase. Once submitted or approved, the data is locked and metrics should not be recalculated.
- `Map<String, BigDecimal> context` — This is the computation context: a map of metric name → computed value. It is built progressively — earlier values (like `Total_Loans`) are used to compute later ones (like `Loan_to_Deposit_Ratio`), and eventually all base values feed into the template formula evaluator.
- `BigDecimal` for all calculations — `double` and `float` have floating-point precision errors (e.g., `0.1 + 0.2 ≠ 0.3`). In banking and regulatory reporting, every cent matters, so `BigDecimal` with `RoundingMode.HALF_UP` is mandatory.
- `riskEvaluator.evaluateFormula(field.getMappingExpression(), context)` — `DynamicRiskEvaluator` is a custom component that parses formula strings like `"Net_GL_Balance / RWA * 100"` and evaluates them against the context map. This makes the system data-driven — you can add new metrics by adding template fields with formulas, without writing Java code.
- `getRiskWeight(loanType)` — Basel III / RBI regulatory framework assigns different risk weights to different loan types. Home loans (secured, lower risk) get 50%, personal/auto loans get 75%, and corporate loans get 100%. These weights are used to compute Risk-Weighted Assets (RWA), which is the denominator for CRAR.
- Regulatory thresholds (checked during exception generation):
    - **CRAR < 9%** — Capital to Risk-weighted Assets Ratio below the RBI minimum of 9%. This means the bank does not have enough capital buffer.
    - **LCR < 100%** — Liquidity Coverage Ratio below 100%. The bank cannot cover 30 days of net cash outflows with high-quality liquid assets.

- **Loan-to-Deposit Ratio > 90%** — Too much of deposits are lent out, leaving little liquidity cushion.
- **Net_GL_Balance < 0** — The net General Ledger balance is negative, indicating potential insolvency.

PHASE 7 — Report Lifecycle (Reporting Officer)

Backend: RegReport.java — Entity

```
@Entity
@Table(name = "RegReport")
@Data
@NoArgsConstructor
public class RegReport {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ReportID")
    private Integer reportId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "TemplateID")
    private RegTemplate template;

    @Column(name = "Period")
    private String period;

    @Column(name = "GeneratedDate")
    private LocalDateTime generatedDate;

    @Column(name = "Status")
    private String status;
}
```

Explanation:
`RegReport` is the central entity of the entire application. Every workflow action in the system ultimately serves to produce, validate, and file a `RegReport`.

- `@ManyToOne(fetch = FetchType.LAZY)` `@JoinColumn(name = "TemplateID")` — Each report is based on one `RegTemplate` (many reports can use the same template). The

template defines which regulatory fields are required and what formulas calculate them. LAZY fetch prevents loading the full template every time a report is queried.

- `period` — The reporting period (e.g., `"2026-Q1"`, `"2026-03"`) to which this report's financial data belongs. This is not the date the report was generated — it is the business period being reported on.
- `generatedDate` — The timestamp when the report was created in the system. Automatically set to `LocalDateTime.now()` in `ReportingService.generateReport()`.
- `status` — The lifecycle state of the report. It is a state machine with exactly four valid states in strict order:
  - `"DRAFT"` — Report has been generated, risk metrics can be calculated.
  - `"UNDER_REVIEW"` — Submitted by COMPLIANCE_ANALYST for admin approval.
  - `"APPROVED"` — Approved by REGTECH_ADMIN, ready for filing.
  - `"FILED"` — Filed with the regulator by REPORTING_OFFICER. Terminal state.

Backend: FilingWorkflow.java — Audit trail per step

```
@Entity
@Table(name = "FilingWorkflow")
@Data
@NoArgsConstructor
public class FilingWorkflow {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "WorkflowID")
    private Integer workflowId;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "ReportID")
    private RegReport report;

    @Column(name = "StepName")
    private String stepName;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "ActorID")
    private User actor;

    @Column(name = "StepDate")
    private LocalDateTime stepDate;
```

```
    @Column(name = "Status")
    private String status;

    @Column(name = "Comments")
    private String comments;
}
```

Explanation:
`FilingWorkflow` records every state transition a report goes through — it is the historical audit trail of the report's lifecycle.

- Every time a report changes status (DRAFT → UNDER_REVIEW → APPROVED → FILED), a new `FilingWorkflow` row is inserted. So for a fully filed report, there will be 4 rows in `FilingWorkflow`, one per step.
- `@ManyToOne @JoinColumn(name = "ReportID")` — Many workflow steps belong to one report.
- `@ManyToOne @JoinColumn(name = "ActorID")` — Records which `User` performed this step. This is critical for accountability — you can always see who submitted, who approved, and who filed the report.
- `stepName` — The name of the new status the report moved into (`"DRAFT"` , `"UNDER_REVIEW"` , `"APPROVED"` , `"FILED"`).
- `comments` — Optional text, most relevant for the `"APPROVED"` step where the admin can leave remarks for the reporting officer.
- `stepDate` — The exact timestamp of the transition. Combined with `actor` , this creates a full chain of custody for the report.

Backend: ReportController.java

```
@RestController
@RequestMapping("/api/report")
public class ReportController {

    @PostMapping("/generate")
    @PreAuthorize("hasAnyRole('REPORTING_OFFICER','REGTECH_ADMIN')")
    public ResponseEntity<RegReportDTO> generateReport(
            @RequestParam(required = false, defaultValue = "1") Integer t
            @RequestParam(required = false, defaultValue = "2026-Q1") Str
        RegReport report = reportingService.generateReport(templateId, pe
        auditService.logAction("GENERATED_REPORT", "Reports",
            "Template #" + templateId + " | Period: " + period + " | Repo
```

```
        notificationService.notifyRole("COMPLIANCE_ANALYST",
            "Draft report #" + report.getReportId() + " ready for review"
        notificationService.notifyRole("RISK_ANALYST",
            "Draft report #" + report.getReportId() + " generated — metri
        return ResponseEntity.ok(modelMapper.map(report, RegReportDTO.cla
    }

    @GetMapping
    @PreAuthorize("hasAnyRole('REPORTING_OFFICER', 'COMPLIANCE_ANALYST',
    public ResponseEntity<List<RegReportDTO>> getAllReports() { ... }

    @GetMapping("/{id}/workflow")
    @PreAuthorize("hasAnyRole('REPORTING_OFFICER', 'COMPLIANCE_ANALYST',
    public ResponseEntity<List<FilingWorkflowDTO>> getWorkflow(@PathVaria

    @PutMapping("/{id}/submit")
    @PreAuthorize("hasAnyRole('COMPLIANCE_ANALYST')")
    public ResponseEntity<?> submitReport(@PathVariable Integer id) { ...

    @PutMapping("/{id}/approve")
    @PreAuthorize("hasRole('REGTECH_ADMIN')")
    public ResponseEntity<?> approveReport(@PathVariable Integer id,
            @RequestParam(required = false) String comments) { ... }

    @PutMapping("/{id}/file")
    @PreAuthorize("hasAnyRole('REPORTING_OFFICER')")
    public ResponseEntity<?> fileReport(@PathVariable Integer id) { ... }
}
```

Explanation:
This controller is the API surface for the entire report lifecycle. Each endpoint maps to one specific step and is locked to the appropriate role.

- `@RequestParam(required = false, defaultValue = "1")` — Makes the query parameter optional; if the frontend doesn't send it, the default value is used. This makes the API more lenient and backwards compatible.
- **Role segregation across endpoints** — This is deliberate role-based workflow enforcement:
  - `generate` — `REPORTING_OFFICER` or `REGTECH_ADMIN` creates the draft.
  - `submit` — Only `COMPLIANCE_ANALYST` can submit for review (they have reviewed the data quality).
  - `approve` — Only `REGTECH_ADMIN` can approve (final authority sign-off).
  - `file` — Only `REPORTING_OFFICER` can file with the regulator (the person responsible for submission).

- `getAllReports` and `getWorkflow` — Multiple roles can read; they just cannot change state.
- Each state-changing endpoint calls `auditService.logAction()` to record the event AND `notificationService.notifyRole()` to alert the next person in the workflow. This creates a push-notification workflow where each actor is informed when it is their turn to act.
- `ResponseEntity<?>` — The `?` wildcard return type is used because the method can return either a `RegReportDTO` (on success) or a plain error string (on exception). This is a pragmatic design choice.

Backend: ReportingService.java — Core state machine

```
@Service
public class ReportingService {

    public RegReport generateReport(Integer templateId, String period) {
        RegReport report = new RegReport();
        report.setTemplate(regTemplateRepository.getReferenceById(templat
        report.setPeriod(period);
        report.setGeneratedDate(LocalDateTime.now());
        report.setStatus("DRAFT");
        report = reportRepository.save(report);

        FilingWorkflow workflow = new FilingWorkflow();
        workflow.setReport(report);
        workflow.setStepName("DRAFT");
        workflow.setActor(getCurrentUser());
        workflow.setStepDate(LocalDateTime.now());
        workflow.setStatus("COMPLETED");
        workflowRepository.save(workflow);
        return report;
    }

    @Transactional
    public RegReport submitReportForReview(Integer reportId) {
        return advanceWorkflow(reportId, "DRAFT", "UNDER_REVIEW", null);
    }

    @Transactional
    public RegReport approveReport(Integer reportId, String comments) {
        return advanceWorkflow(reportId, "UNDER_REVIEW", "APPROVED", comm
    }
```

```
    @Transactional
    public RegReport fileReport(Integer reportId) {
        return advanceWorkflow(reportId, "APPROVED", "FILED", null);
    }

    private RegReport advanceWorkflow(Integer reportId, String expectedCu
                                      String nextStatus, String comments
        RegReport report = reportRepository.findById(reportId)
            .orElseThrow(() -> new ResourceNotFoundException("Report not

        if (!expectedCurrentStatus.equals(report.getStatus())) {
            throw new ValidationException(
                "Cannot advance report to " + nextStatus
                + ". Current status is " + report.getStatus()
                + ", expected " + expectedCurrentStatus);
        }

        report.setStatus(nextStatus);
        report = reportRepository.save(report);

        FilingWorkflow workflow = new FilingWorkflow();
        workflow.setReport(report);
        workflow.setStepName(nextStatus);
        workflow.setActor(getCurrentUser());
        workflow.setStepDate(LocalDateTime.now());
        workflow.setStatus("COMPLETED");
        if (comments != null && !comments.trim().isEmpty()) workflow.setC
        workflowRepository.save(workflow);

        auditService.logAction("WORKFLOW_ADVANCE", "ReportID: " + reportI
            "Status changed from " + expectedCurrentStatus + " to " + nex
        return report;
    }

    private User getCurrentUser() {
        Authentication auth = SecurityContextHolder.getContext().getAuthe
        if (auth != null && auth.isAuthenticated() && !"anonymousUser".eq
            return userRepository.findByUsername(auth.getName()).orElse(r
        }
        return null;
    }
}
```

Explanation:

`ReportingService` implements the report lifecycle as a strict state machine through the single private `advanceWorkflow` method.

- `regTemplateRepository.getReferenceById(templateId)` — Unlike `findById()` which issues a SELECT, `getReferenceById()` returns a Hibernate proxy object with only the ID set, without hitting the database. This is safe here because we only need the template FK reference for the `RegReport` FK column.
- `advanceWorkflow(reportId, expectedCurrentStatus, nextStatus, comments)` — This is the single choke point for all status changes. It enforces the state machine:
  1. Loads the report by ID (throws `ResourceNotFoundException` if missing).
  2. Validates that the report is in the expected current status. If someone tries to approve a report that is still `"DRAFT"` (skipping the submit step), `ValidationException` is thrown and caught by `GlobalExceptionHandler` as a 400 Bad Request. This is the state machine enforcement.
  3. Updates the status.
  4. Creates a new `FilingWorkflow` row to permanently record this transition with the actor's identity and timestamp.
  5. Logs to `AuditLog`.
- `getCurrentUser()` — Reads the `Authentication` from `SecurityContextHolder` (put there by `JwtAuthenticationFilter`) and looks up the full `User` entity from the DB. Returns `null` if called in an unauthenticated context (defensive programming).
- `"anonymousUser".equals(auth.getPrincipal())` — Spring Security uses the string `"anonymousUser"` as the principal for unauthenticated requests (those that passed `permitAll()`). Checking this prevents a DB lookup for anonymous access.

Frontend: report.service.ts

```
@Injectable({ providedIn: 'root' })
export class ReportService {
  private readonly API_URL = 'http://localhost:9090/api/report';

  constructor(private http: HttpClient) {}

  getReports(): Observable<RegReport[]> {
    return this.http.get<RegReport[]>(this.API_URL);
  }

  generateReport(templateId: number, period: string): Observable<RegRepor
```

```
    const params = new HttpParams()
      .set('templateId', templateId.toString())
      .set('period', period);
    return this.http.post<RegReport>(`${this.API_URL}/generate`, null, {
  }

  submitReport(id: number): Observable<RegReport> {
    return this.http.put<RegReport>(`${this.API_URL}/${id}/submit`, null)
  }

  approveReport(id: number, comments?: string): Observable<RegReport> {
    const params = comments
      ? new HttpParams().set('comments', comments)
      : new HttpParams();
    return this.http.put<RegReport>(`${this.API_URL}/${id}/approve`, null
  }

  fileReport(id: number): Observable<RegReport> {
    return this.http.put<RegReport>(`${this.API_URL}/${id}/file`, null);
  }

  getWorkflow(id: number): Observable<FilingWorkflowStep[]> {
    return this.http.get<FilingWorkflowStep[]>(`${this.API_URL}/${id}/wor
  }
}
```

Explanation:

`ReportService` is the Angular service that wraps all HTTP calls related to reports. It is the frontend's interface to the `ReportController` on the backend.

- Every method returns an `Observable<T>` — Angular's `HttpClient` is Observable-based (RxJS). The Observable is cold (lazy): no HTTP request is sent until something calls `.subscribe()`. The component calls `.subscribe({ next, error })` to trigger the request and handle the response.
- `HttpParams` — Immutable query string builder. `.set('key', 'value')` returns a new `HttpParams` object (does not mutate). Used for `?templateId=1&period=2026-Q1` on the generate endpoint.
- `http.post(..., null, { params })` — The second argument is the request body (null because the backend reads params from query string, not body). The third argument is request options including the query params.
- `comments?: string` — The `?` makes `comments` an optional parameter in TypeScript. If not provided, no `comments` query param is sent, and the backend's `@RequestParam(required`

`= false)` handles the absence gracefully.

- `http.put(url, null)` — HTTP PUT with a null body. Spring's `@PutMapping` on these endpoints does not expect a request body — the report ID is in the URL path and the action is implicit (submit/approve/file).

PHASE 8 — Audit & Notifications

Backend: AuditService.java

```
@Service
public class AuditService {

    private final AuditLogRepository auditLogRepository;
    private final UserRepository userRepository;

    public AuditLog logAction(String action, String resource, String meta
        AuditLog log = new AuditLog();
        log.setAction(action);
        log.setResource(resource);
        log.setTimestamp(LocalDateTime.now());
        log.setMetadata(metadata);

        try {
            Authentication auth = SecurityContextHolder.getContext().getA
            if (auth != null && auth.isAuthenticated()
                    && !auth.getPrincipal().equals("anonymousUser")) {
                String username = auth.getName();
                Optional<User> userOpt = userRepository.findByUsername(us
                if (userOpt.isPresent()) {
                    log.setUser(userOpt.get());
                } else {
                    userRepository.findByEmail(username).ifPresent(log::s
                }
            }
        } catch (Exception e) {
            // Audit should never block business logic
        }

        return auditLogRepository.save(log);
    }
}
```

Explanation:
`AuditService` is a cross-cutting concern — it is called from controllers, services, and filters throughout the application to create a tamper-evident activity log.

- `logAction(String action, String resource, String metadata)` — Three-parameter overload. `action` is a short code like `"FILED_REPORT"`, `resource` is what was acted on (e.g., `"Reports"`), and `metadata` is additional context (e.g., `"Report #5 filed with regulator"`).
- The `try/catch` around the user resolution is intentional and important: `auditService.logAction()` must never throw an exception because it is called inside business logic methods (like `fileReport()`). If the audit fails, the business operation should still complete. The pattern "audit should never block business logic" is a design principle here.
- `auth.getPrincipal().equals("anonymousUser")` — Guards against the edge case where audit is called in an anonymous context (e.g., during application startup initializers).
- Fallback `findByEmail(username)` — Some authentication configurations use email as the principal instead of username. The fallback handles both cases.
- Every `AuditLog` record contains: action, resource, metadata, user (who did it), and timestamp (when). This data is surfaced in the admin's audit log view on the frontend.

Backend: NotificationService.java

```
@Service
public class NotificationService {

    public void notifyRole(String role, String message, String category)
        List<User> users = userRepository.findByRole(role);
        for (User user : users) {
            if (!"ACTIVE".equalsIgnoreCase(user.getStatus())) continue;
            createNotification(user, message, category);
        }
    }

    private void createNotification(User user, String message, String cat
        Notification notification = new Notification();
        notification.setUser(user);
        notification.setMessage(message);
        notification.setCategory(category);
        notification.setStatus("UNREAD");
        notification.setCreatedDate(LocalDateTime.now());
```

```
        notificationRepository.save(notification);
    }

    public List<Notification> getMyNotifications() {
        Long userId = getCurrentUserId();
        if (userId == null) return List.of();
        return notificationRepository.findByUser_IdOrderByCreatedDateDesc
    }

    public long getUnreadCount() {
        Long userId = getCurrentUserId();
        if (userId == null) return 0;
        return notificationRepository.countByUser_IdAndStatus(userId, "UN
    }

    @Transactional
    public void markAllAsRead() {
        Long userId = getCurrentUserId();
        if (userId != null) {
            notificationRepository.markAllReadByUserId(userId);
        }
    }
}
```

Explanation:

`NotificationService` implements an in-app notification system that keeps all workflow actors informed of events relevant to them.

- `notifyRole(String role, String message, String category)` — Finds ALL active users with the specified role and creates a notification for each. For example, when a report is approved, all `REPORTING_OFFICER` users are notified — not just one specific person.
- `if (!"ACTIVE".equalsIgnoreCase(user.getStatus())) continue` — Skips inactive/deactivated users. Sending notifications to inactive accounts would be noise.
- `createNotification(...)` — Persists a `Notification` entity to the database with status `"UNREAD"`. Notifications are pull-based (not WebSockets/push) — the frontend polls or queries for them.
- `category` — A label like `"Data Upload"`, `"Ingestion"`, `"Risk"`, `"Report"`, `"Data Quality"` that the frontend can use to categorize or filter notifications.
- `getMyNotifications()` — Returns only the calling user's own notifications, ordered newest-first. Uses `getCurrentUserId()` which reads from `SecurityContextHolder` — so users can only see their own notifications.

- `markAllAsRead()` — Bulk-updates all of the current user's `UNREAD` notifications to `READ`. The `@Transactional` ensures all updates are committed atomically.

PHASE 9 — Exception Handling (Global)

Backend: GlobalExceptionHandler.java

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleResourceNotFoundExce
            ResourceNotFoundException ex) {
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.NOT_FOUND.value());
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());
        return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidFileException.class)
    public ResponseEntity<Map<String, Object>> handleInvalidFileExceptior
            InvalidFileException ex) {
        // → HTTP 400 Bad Request
    }

    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<Map<String, Object>> handleValidationException(
            ValidationException ex) {
        // → HTTP 400 Bad Request
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGlobalException(Exce
        // → HTTP 500 Internal Server Error (catch-all)
    }
}
```

Explanation:

`GlobalExceptionHandler` is a centralized error handler that intercepts exceptions thrown anywhere in any `@RestController` and converts them to structured JSON error responses.

- `@ControllerAdvice` — This annotation tells Spring to apply this class as a global interceptor for all controllers. Without it, unhandled exceptions would return Spring's default white-label HTML error page, which is not useful for a REST API consumed by Angular.
- `@ExceptionHandler(SomeException.class)` — Maps a specific exception type to a handler method. When `ResourceNotFoundException` is thrown anywhere in the controller layer (or service layer if the exception propagates up), Spring calls `handleResourceNotFoundException()`.
- `ResourceNotFoundException` → HTTP 404: thrown when `repository.findById()` returns empty (`orElseThrow()`). Signals that the requested resource (report, user, issue) does not exist.
- `ValidationException` → HTTP 400: thrown when business rules are violated (e.g., trying to approve a `DRAFT` report, resolving an already-resolved issue). The message explains what went wrong.
- `InvalidFileException` → HTTP 400: thrown by `CsvUploadService` when the uploaded file is not a valid CSV or has wrong columns.
- `Exception.class` (catch-all) → HTTP 500: catches any unexpected exception that slips through. Returns `"Internal Server Error"` to avoid exposing stack traces to the API consumer.
- The response body includes `timestamp`, `status`, `error`, and `message` — a consistent structure that the Angular frontend can always depend on for error display.

Backend: JwtAuthenticationEntryPoint.java

```
@Component
public class JwtAuthenticationEntryPoint implements AuthenticationEntryPo

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse
                         AuthenticationException authException)
            throws IOException, ServletException {
        response.setContentType("application/json");
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.getWriter().write(
            "{\"error\": \"Unauthorized\", \"message\": \""
            + authException.getMessage() + "\"}");
```

```
    }
}
```

Explanation:
This class handles the specific case where a request reaches a protected endpoint without a valid JWT token.

- `implements AuthenticationEntryPoint` — Spring Security calls `commence()` when an unauthenticated request hits a URL that requires authentication (i.e., it is NOT in `permitAll()`). It is registered in `SecurityConfig` via `.authenticationEntryPoint(jwtAuthenticationEntryPoint)`.
- Without this class, Spring Security would return a `302 redirect` to a login form (the default for web applications). But this is a REST API — the frontend does not want an HTML redirect; it wants a `401 JSON` response it can handle programmatically.
- `response.setStatus(HttpServletResponse.SC_UNAUTHORIZED)` — Sets the HTTP status to 401.
- `response.getWriter().write(...)` — Writes a raw JSON string directly to the response body. This is intentionally low-level because `@ExceptionHandler` methods in `@ControllerAdvice` do not intercept security-layer exceptions — those exceptions happen before the request even reaches the controller.
- The Angular `authInterceptor` catches this `401` and forces the user to re-login.

Complete End-to-End Flow Summary

```
User enters username/password in the browser
        ↓
[login.ts] onSubmit() validates fields → calls AuthService.login()
        ↓ POST /api/auth/login  (no token required — permitAll in Securit
[AuthController] receives @RequestBody AuthRequest
        → authenticationManager.authenticate(UsernamePasswordAuthenticati
        → DaoAuthenticationProvider calls CustomUserDetailsService.loadUs
        → UserRepository.findByUsername() → queries MySQL users table
        → BCryptPasswordEncoder.matches(rawPassword, storedHash)
        ↓ Credentials valid
[AuthController] checks user.status == "ACTIVE" (business-level gate)
        ↓ User is active
[JwtService] generateToken(username, role)
        → Jwts.builder().setSubject(username).claim("role", role)
           .setExpiration(now + 10h).signWith(HS256).compact()
```

```
        ↓ Returns { "token": "eyJ..." }
[AuthService] localStorage.setItem('jwt_token', token)
[LoginComponent] getRole() → decodes JWT payload → reads 'role' claim
        → router.navigate(['/reporting']) (or role-specific route)

━━━━━ EVERY SUBSEQUENT HTTP REQUEST ━━━━━

[authInterceptor] reads token from localStorage
        → req.clone({ Authorization: "Bearer <token>" })
        ↓
[JwtAuthenticationFilter.doFilterInternal()]
        → reads Authorization header → extracts token after "Bearer "
        → jwtService.extractUsername(token) → decodes 'sub' claim
        → userDetailsService.loadUserByUsername(username) → DB lookup
        → jwtService.validateToken(token, userDetails) → checks username
        → SecurityContextHolder.setAuthentication(authToken)
        ↓
[@PreAuthorize / SecurityFilterChain]
        → checks GrantedAuthority ("ROLE_REPORTING_OFFICER", etc.)
        → allows or returns 403 via CustomAccessDeniedHandler

━━━━━ OPERATIONS OFFICER ━━━━━

POST /api/ingestion/csv/deposits/upload (multipart/form-data, file field)
        → @PreAuthorize("hasAnyRole('OPERATIONS_OFFICER','REGTECH_ADMIN')
        → CsvUploadService.uploadDeposits(file) → parse CSV → save Deposi
        → NotificationService.notifyRole("COMPLIANCE_ANALYST", "New depos

POST /api/ingestion/run
        → DataIngestionService.runIngestion()
        → loops DataSource records → counts Loan/Deposit/GL/Treasury rows
        → saves RawDataBatch per source (status="Ingested")
        → AuditService.logAction("RUN_INGESTION", ...)
        → NotificationService.notifyRole("COMPLIANCE_ANALYST", "Ingestior

━━━━━ COMPLIANCE ANALYST ━━━━━

GET /api/data-quality/issues
        → DataQualityService.getOpenIssues() → findByStatus("Open")
        → Returns list of DataQualityIssueDTO

PUT /api/data-quality/issues/{id}/resolve  { correctedValue, justificatic
        → @Transactional
        → patch RawRecord payloadJson (ObjectMapper → ObjectNode → put fi
```

```
        → issue.setStatus("Resolved") → save
        → new CorrectionLog (oldValue, newValue, correctedByUser, correct
        → AuditLog → Notifications to COMPLIANCE_ANALYST + REPORTING_OFFI

━━━━━ RISK ANALYST ━━━━━

POST /api/risk/calculate/{reportId}
        → RiskCalculationService.calculateMetrics(reportId)
        → deletes old metrics for this report
        → aggregates Loan/Deposit/GL/Treasury data into context Map
        → loops TemplateField formulas → DynamicRiskEvaluator.evaluateFor
        → saves RiskMetric per field (CRAR, LCR, Loan_to_Deposit_Ratio, e
        → AuditLog → Notifications to COMPLIANCE_ANALYST + REPORTING_OFFI

━━━━━ REPORTING OFFICER — Report Lifecycle ━━━━━

POST /api/report/generate?templateId=1&period=2026-Q1
        → ReportingService.generateReport()
        → new RegReport { status="DRAFT" } → save
        → new FilingWorkflow { stepName="DRAFT", actor=currentUser } → sa
        → AuditLog("GENERATED_REPORT") → Notify COMPLIANCE_ANALYST + RISK
        ← Returns RegReportDTO

PUT /api/report/{id}/submit  [called by COMPLIANCE_ANALYST]
        → advanceWorkflow("DRAFT" → "UNDER_REVIEW")
        → report.status = "UNDER_REVIEW" → save
        → new FilingWorkflow { stepName="UNDER_REVIEW", actor=currentUser
        → AuditLog("SUBMITTED_REPORT") → Notify REGTECH_ADMIN

PUT /api/report/{id}/approve?comments=...  [called by REGTECH_ADMIN]
        → advanceWorkflow("UNDER_REVIEW" → "APPROVED")
        → report.status = "APPROVED" → save
        → new FilingWorkflow { stepName="APPROVED", actor=currentUser, co
        → AuditLog("APPROVED_REPORT") → Notify REPORTING_OFFICER + COMPLI

PUT /api/report/{id}/file  [called by REPORTING_OFFICER]
        → advanceWorkflow("APPROVED" → "FILED")
        → report.status = "FILED" → save
        → new FilingWorkflow { stepName="FILED", actor=currentUser } → sa
        → AuditLog("FILED_REPORT") → Notify REGTECH_ADMIN + COMPLIANCE_AN

GET /api/report/{id}/workflow
        → workflowRepository.findByReport_ReportIdOrderByStepDateAsc(id)
```

```
→ Returns all FilingWorkflow steps (DRAFT, UNDER_REVIEW, APPROVEI
  with actor names, timestamps, and comments
```

Key Technology Keywords Index

Keyword	Where Used	Meaning
`@SpringBootApplication`	Main class	Combines @Configuration + @EnableAutoConfiguration + @ComponentScan
`@RestController`	All controllers	JSON REST endpoint — @Controller + @ResponseBody
`@RequestMapping`	Controllers	Base URL path prefix for all methods in the class
`@GetMapping` / `@PostMapping` / `@PutMapping`	Controllers	Maps HTTP verb + path to a method
`@PathVariable`	Controllers	Binds URL segment `{id}` to a method parameter
`@RequestParam`	Controllers	Binds URL query parameter `?key=val` to a method parameter
`@RequestBody`	Controllers	Deserializes HTTP request JSON body to a Java object
`ResponseEntity<T>`	Controllers	Wraps return value with HTTP status code and headers
`@PreAuthorize`	Methods/Classes	SpEL expression evaluated before method executes — role-based access
`@ControllerAdvice`	GlobalExceptionHandler	Global exception interceptor for all @RestControllers
`@ExceptionHandler`	GlobalExceptionHandler	Maps specific exception class to a handler method
`@Entity` / `@Table`	Models	JPA entity — maps Java class to a DB table
`@Id` / `@GeneratedValue`	Models	Primary key + auto-increment strategy
`@Column`	Models	Maps field to DB column, can set constraints like unique=true
`@ManyToOne` / `@JoinColumn`	Models	Many-to-one FK relationship

Keyword	Where Used	Meaning
`FetchType.LAZY`	Models	Related entity loaded on-demand (prevents N+1 queries)
`@PrePersist`	Models	JPA lifecycle hook — runs before INSERT
`@Data / @NoArgsConstructor`	Models	Lombok — generates boilerplate (getters, setters, constructors)
`@Transactional`	Services	All DB operations in method are one atomic transaction
`@PostConstruct`	Services	Runs after Spring finishes dependency injection
`@Service`	Services	Spring stereotype — marks as a managed service bean
`@Component`	Filters/Handlers	Spring stereotype — marks as a managed component bean
`@Value("${...}")`	JwtService	Injects value from application.properties
`@Bean`	SecurityConfig	Declares a method that produces a Spring-managed bean
`@Configuration`	SecurityConfig	Class is a source of @Bean definitions
`@EnableWebSecurity`	SecurityConfig	Activates Spring Security web support
`@EnableMethodSecurity`	SecurityConfig	Enables @PreAuthorize on methods
`@Autowired`	Various	Spring injects the matching bean (field/constructor/setter)
`JpaRepository<T, ID>`	Repositories	Provides CRUD + Spring Data derived queries
`Optional<T>`	Repositories/Services	Container that may or may not hold a value — forces null handling
`SecurityContextHolder`	Services/Filters	Thread-local store for the current authenticated principal
`OncePerRequestFilter`	JwtAuthenticationFilter	Servlet filter that executes exactly once per request
`FilterChain`	JwtAuthenticationFilter	Chain of filters — must call doFilter() to continue
`BCryptPasswordEncoder`	SecurityConfig	Adaptive password hashing — secure one-way algorithm

Keyword	Where Used	Meaning
`SessionCreationPolicy.STATELESS`	SecurityConfig	No HttpSession — each request is self-contained
`UsernamePasswordAuthenticationToken`	Filter/Controller	Spring Security's standard authentication object
`SimpleGrantedAuthority`	UserDetailsService	Wraps role string as a Spring Security permission
`DaoAuthenticationProvider`	SecurityConfig	Loads user from DB + verifies BCrypt password
`ModelMapper`	Controllers	Converts JPA entities to DTOs and vice versa
`BigDecimal`	RiskCalculationService	Exact decimal arithmetic — mandatory for financial calculations
`RoundingMode.HALF_UP`	RiskCalculationService	Standard banker's rounding
`ObjectMapper / ObjectNode`	DataQualityService	Jackson JSON parser/manipulator
`Observable<T>`	Angular services	RxJS lazy async stream — nothing happens until subscribed
`tap`	AuthService	RxJS side-effect operator — observes without modifying the stream
`catchError / throwError`	authInterceptor	RxJS error handling operators
`HttpInterceptorFn`	authInterceptor	Functional HTTP interceptor (Angular 15+)
`CanActivateFn`	Guards	Functional route guard type (Angular 15+)
`standalone: true`	All components	Component without NgModule — self-contained
`loadComponent`	app.routes.ts	Lazy-loads component bundle on first navigation
`inject()`	Guards/Interceptors	DI outside class constructor — functional injection context
`provideHttpClient`	app.config.ts	Registers HttpClient + interceptors globally
`withInterceptors([...])`	app.config.ts	Attaches functional interceptors to HttpClient
`localStorage`	AuthService	Browser key-value storage — persists JWT across page refreshes

Keyword	Where Used	Meaning
`atob()`	AuthService	Native browser Base64 decoder — used to read JWT payload
`HttpParams`	report.service.ts	Immutable URL query string builder
`req.clone({...})`	authInterceptor	Creates modified copy of immutable HttpRequest